DESENVOLVIMENTO DE SOFTWARE COM C#

Aula 16: Consumo de APIs e JSON em C#

Nesta aula, vamos explorar como conectar suas aplicações C# ao mundo exterior — consumindo serviços externos, interpretando dados estruturados e dominando as ferramentas essenciais do desenvolvedor moderno.



FUNDAMENTOS

O Que é uma API REST?

Uma **API (Application Programming Interface)** é uma interface que permite a comunicação entre diferentes sistemas de software. Quando falamos em **REST** (Representational State Transfer), estamos nos referindo a uma arquitetura baseada no protocolo **HTTP**, amplamente adotada por serviços como GitHub, Spotify e Google Maps.

URLs como Recursos

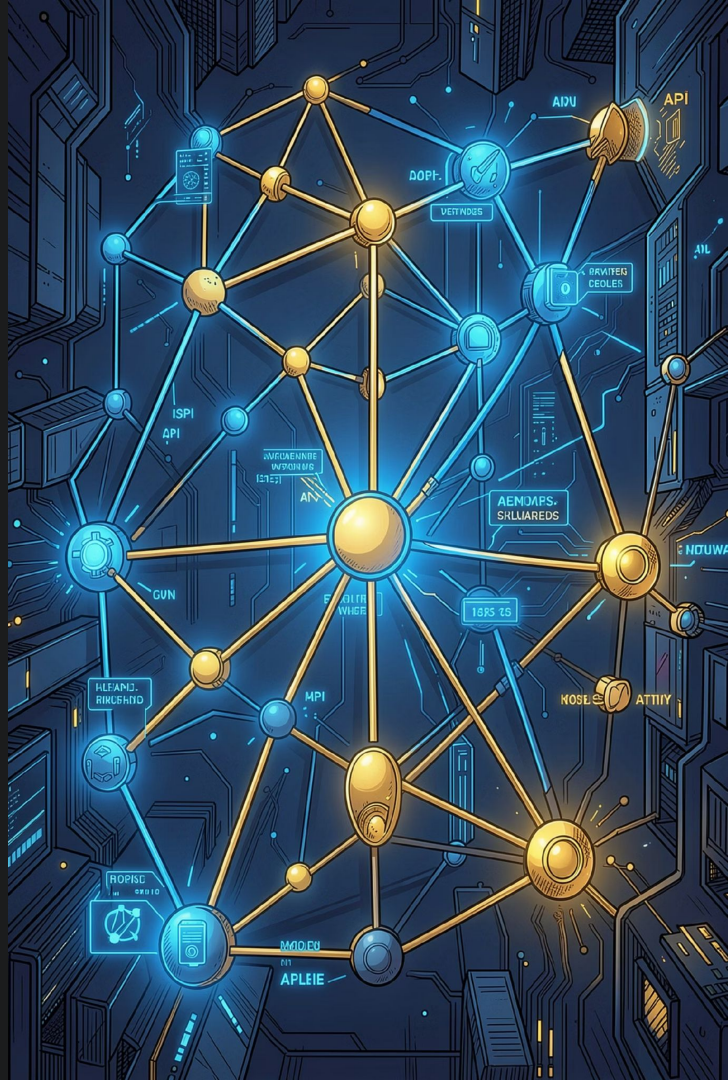
Cada recurso é identificado por uma URL única, como `/api/usuarios/1`, tornando a API intuitiva e previsível.

Verbos HTTP

GET para ler, **POST** para criar, **PUT** para atualizar e **DELETE** para remover dados no servidor.

Stateless

Cada requisição contém todas as informações necessárias — o servidor não guarda estado entre as chamadas.



FUNDAMENTOS

A Anatomia do HTTP

A comunicação via HTTP segue um modelo simples: o **cliente** envia uma requisição e o **servidor** retorna uma resposta. Entender essa troca é essencial para diagnosticar problemas e construir integrações robustas.



Cliente envia

Servidor processa

Servidor responde

O diagrama acima ilustra o fluxo completo de uma comunicação HTTP: desde o envio da requisição pelo cliente até a resposta final do servidor, passando pelo processamento e validação dos dados.

Códigos de Status

- **2xx** — Sucesso (ex: 200 OK)
- **3xx** — Redirecionamento
- **4xx** — Erro do cliente (ex: 404)
- **5xx** — Erro do servidor (ex: 500)

Componentes da Requisição

- **Método**: GET, POST, PUT, DELETE
- **URL**: endereço do recurso
- **Headers**: metadados (auth, content-type)
- **Body**: dados enviados (em POST/PUT)

JSON

JSON: A Linguagem da Web

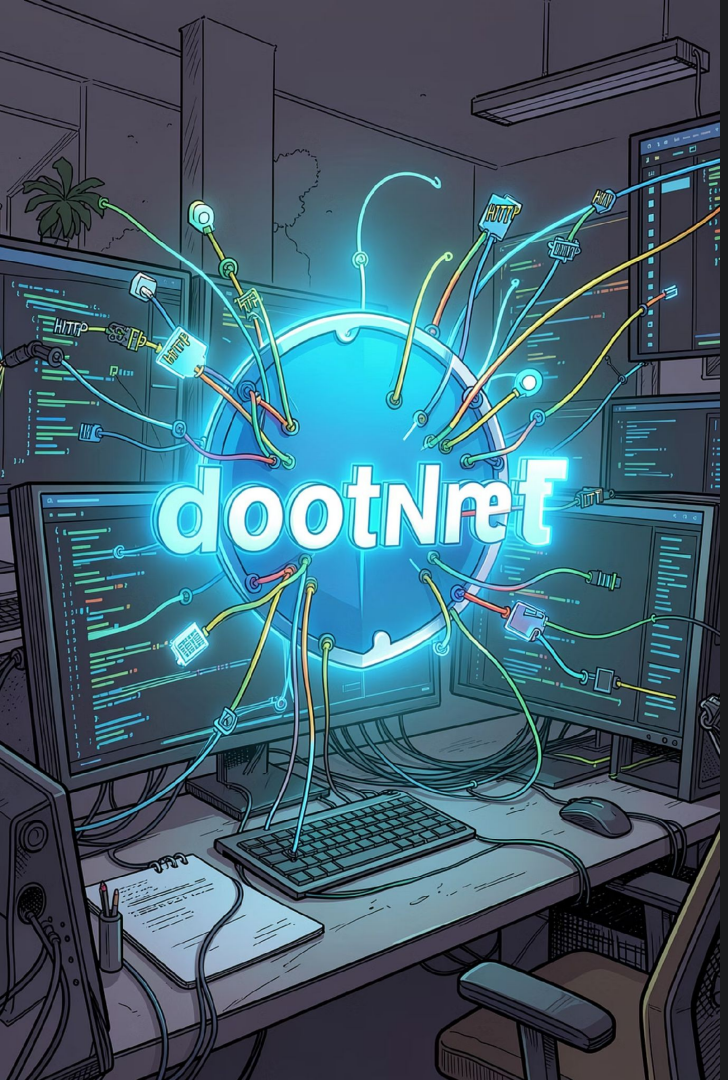
JSON (JavaScript Object Notation) é o formato padrão para troca de dados em APIs REST. Leve, legível por humanos e independente de linguagem, ele se tornou o *lingua franca* da internet moderna.

Estrutura Básica

```
{
  "id": 1,
  "nome": "Maria Silva",
  "ativo": true,
  "roles": ["admin", "dev"],
  "endereco": {
    "cidade": "São Paulo",
    "cep": "01000-000"
  }
}
```

Tipos de Dados JSON

- **String** — Texto entre aspas
- **Number** — Inteiros e decimais
- **Boolean** — `true` ou `false`
- **Array** — Lista ordenada
- **Object** — Coleção chave-valor



HTTPCLIENT

O Papel do HttpClient no .NET

O **HttpClient** é a classe nativa do .NET para enviar solicitações HTTP e receber respostas. É o ponto de entrada para qualquer integração externa em C#, desde consumir APIs públicas até conectar-se a microsserviços internos.

Instanciação Correta

Evite criar novas instâncias a cada requisição. Use `IHttpClientFactory` para gerenciar o ciclo de vida e prevenir esgotamento de sockets.

Métodos Principais

`GetAsync()`, `PostAsync()`, `PutAsync()` e `DeleteAsync()` cobrem todos os verbos HTTP necessários para integrar com APIs REST.

Exemplos Práticos

Serviços como **GitHub API**, **JSONPlaceholder** e **OpenWeatherMap** são excelentes pontos de partida para praticar o consumo de APIs reais.

SERIALIZAÇÃO

Serialização e Desserialização

Para que o C# entenda os dados recebidos de uma API, é necessário **converter JSON em objetos C#** (desserialização) e, ao enviar dados, **converter objetos em JSON** (serialização). O namespace `System.Text.Json` oferece performance superior ao antigo `Newtonsoft.Json`.

🔄 Serialização

Objeto C# → String JSON

Usado ao **enviar dados** para a API em requisições POST/PUT.

```
var json = JsonSerializer.Serialize(usuario);
```

📥 Desserialização

String JSON → Objeto C#

Usado ao **receber dados** da API e transformá-los em objetos utilizáveis.

```
var usuario =
JsonSerializer.Deserialize<Usuario>(json);
```

ⓘ 💡 O `System.Text.Json` foi introduzido no .NET Core 3.0 e é mantido pela Microsoft como substituto de alta performance do Newtonsoft.Json.

MÃO NA MASSA

HttpClient em Ação: Código Real

Veja como consumir um endpoint real usando **HttpClient** e **System.Text.Json** em poucos passos. Este padrão se repete em praticamente qualquer integração com API REST.

Exemplo Completo

```
using var client = new HttpClient();
var response = await client.GetAsync(
    "https://api.github.com/users/dotnet");

response.EnsureSuccessStatusCode();

var json = await response.Content
    .ReadAsStringAsync();

var user = JsonSerializer.Deserialize<GithubUser>(json);

Console.WriteLine($"Nome: {user.Name}");
Console.WriteLine($"Repos: {user.PublicRepos}");
```

Passo a Passo

01

Instanciar o HttpClient

Abre a conexão com o servidor remoto.

02

Chamar GetAsync

Envia a requisição GET para o endpoint.

03

Validar com EnsureSuccessStatusCode

Lança exceção se o status for erro (4xx/5xx).

04

Desserializar o JSON

Converte o corpo da resposta em objeto C#.

SYSTEM.TEXT.JSON

Otimizando com System.Text.Json

Além da desserialização básica, o `System.Text.Json` oferece recursos avançados como **geração de código-fonte em tempo de compilação** (source generators), que elimina reflexão e melhora drasticamente a performance em aplicações de alta demanda.



Source Generators

Com `[JsonSerializable]`, o compilador gera código otimizado, eliminando a reflexão em tempo de execução e reduzindo alocações de memória.



Mapeamento de Propriedades

Use `[JsonPropertyName("campo_json")]` para mapear nomes de campos JSON diferentes dos nomes das propriedades C#.



JsonSerializerOptions

Configure comportamento global: `PropertyNameCaseInsensitive`, `DefaultIgnoreCondition` e formatos de data personalizados.



Leitura Assíncrona

Use `DeserializeAsync` com streams para desserializar grandes payloads sem bloquear a thread principal da aplicação.

BOAS PRÁTICAS

Boas Práticas de Integração com APIs

Consumir APIs de forma profissional vai além de fazer a chamada funcionar. É preciso tratar erros, organizar o código e garantir que a aplicação seja resiliente e escalável.



Tratamento de Erros

Sempre envolva chamadas HTTP em `try-catch`. Trate `HttpRequestException`, `JsonException` e valide códigos de status antes de desserializar.



Uso do LINQ

Após desserializar coleções, utilize `.Where()`, `.Select()` e `.OrderBy()` para filtrar e transformar os dados antes de exibi-los na interface.



Separação de Responsabilidades

Isolar a lógica de acesso à API em uma **camada de serviço** (ex: `IApiService`) torna o código testável, substituível e independente da interface do usuário.



Reutilize o HttpClient

Registre o `HttpClient` via `IHttpClientFactory` na injeção de dependência para evitar problemas de esgotamento de portas TCP.

CONCLUSÃO

Conclusão: Conectando o Mundo

Nesta aula, construímos a base para integrar suas aplicações C# com qualquer serviço externo. APIs são as **pontes que expandem o poder** das suas aplicações — e dominar HTTP, JSON e HttpClient é o diferencial do desenvolvedor profissional.

4

Verbos HTTP

GET, POST, PUT e DELETE cobrem a maioria das operações em APIs REST.

2

Direções de Conversão

Serialização (objeto → JSON) e Desserialização (JSON → objeto).

1

Próximo Passo

Integrar sua primeira API real em um projeto prático e funcional.

🚀 **Desafio Prático:** Crie um projeto console em C# que consuma a API do **JSONPlaceholder** (`https://jsonplaceholder.typicode.com/posts`), desserialize os dados em uma lista de objetos e exiba os títulos dos primeiros 5 posts usando LINQ.

